Lab Tasks

Task # 01:

Design an abstract class called Vehicle to represent different types of vehicles. This class should have three properties: make (a string for the vehicle's brand), model (a string for the vehicle's model), and year (an integer for the year it was manufactured). The class should also declare an abstract method getSalePrice() that returns the sale price of the vehicle. Then, create two concrete classes, Car and Truck, that inherit from Vehicle. The Car class should include a constructor that initializes the make, model, year, and base price of the car. The Truck class should include a constructor that initializes the make, model, year, base price, and weight capacity of the truck. Implement the getSalePrice() method in both classes: for a car, the sale price is the base price multiplied by a coefficient based on the car's age (1.2 if less than 3 years old, 0.9 if 3–10 years old, 0.5 if older than 10 years); for a truck, the sale price is the base price plus a premium based on its weight capacity (5000 for less than 2 tons, 10000 for 2–5 tons, 20000 for more than 5 tons). Finally, create Car and Truck objects using Vehicle class references and calculate and print their sale prices using the getSalePrice() method.

Task # 02:

Define an abstract class **Character** that contains:

- an integer health variable that represents the character's health,
- an integer attackPower variable that represents the character's attack power
- string name variable that represents the character's name.
- specialAbility() (abstract method): An abstract method that must be implemented by the subclass for each character type. This method defines the unique special ability of each character type.
- attack() (non-abstract method): A method that takes another Character object as a parameter and simulates an attack from the current character to the target character. It subtracts the attack power of the current character from the target character's health.

Define **Warrior** class that contains:

- specialAbility() (method implementation): Overrides the abstract specialAbility() method in the Character class to define the unique special ability for a Warrior character. In this implementation, the warrior's attack power is doubled.

Define **Mage** class that contains:

- specialAbility() (method implementation): Overrides the abstract specialAbility() method in the Character class to define the unique special ability for a Mage character. In this implementation, the mage's attack power is increased by 20.

Define class **Archer** that contains:

- specialAbility() (method implementation): Overrides the abstract specialAbility() method in the Character class to define the unique special ability for an Archer character. In this implementation, the archer's attack power is increased by 15.

Finally, create an instance of each subclass in Game(Main Class) class and set their attributes such as name, health, and attackPower. Then call the attack() method to simulate a battle between the characters. Also call the specialAbility() method to use the unique special ability of each character.

Task # 03:

Create a class named Car with two attributes: carName and carType. Include a parameterized constructor to set these attributes, and a private method getCarName() that returns the car name. Then create another class named Engine as an inner class inside Car. The Engine class should have an attribute engineType and a method setEngine() that sets the engine type based on the following rules: if the car type is "4T", check the car name—if it is "Mehran", set the engine type to "small"; otherwise set it to "large". If the car type is not "4T", set the engine type to "Bigger". The Engine class should also have a method getEngineType() to return the engine type. In the main program, create an object of the outer class Car and then create an object of the inner class Engine using the outer class object. Call the appropriate methods to set and get the engine type.

Task # 04:

Create a class called Vehicle with three attributes: vehicleName, engineCC, and model. The class should have a method display() that lists all the attributes of the vehicle. Inside the display() method, define a local inner class named Owner with attributes ownerName, CNIC, and phoneNumber. Provide proper constructors, setters, and getters for both the Vehicle class and the Owner class. In the main program, create an object of the Vehicle class using an anonymous inner class and override the display() method to show all the details of the vehicle along with its owner's information (vehicle name, engine CC, model, owner name, and CNIC). Assign an owner to the vehicle and ensure all details are displayed when calling the overridden display() method.